Assignment: Textual Tour of OC CS Speckit

Goal: Understand Spec-Driven Development (SDD), what **OC CS Speckit** is, and what the major folders and files in this repository mean. This is a **reading / orientation** tour — you are not asked to install, rebuild, or ship code here.

Open the repo in Cursor and follow the sections below, opening linked files as you go.

Hands-on assignments (separate):

Assignment	Purpose
ASSIGNMENT-rebuild-todo.md	Strip Todo answer-key code and rebuild from specs
ASSIGNMENT-starter-kit.md	Start a **new** product from the starter kit zip

---

1. What is Spec-Driven Development (SDD)?

In many projects, people write code first and document later (or never). **Spec-Driven Development** flips that:

1. **Write the spec first** — user stories, functional requirements (**FR-00N**), success criteria (**SC-00N**), API/data contracts, screens, and Gherkin acceptance scenarios live in Markdown under `features/` *before* (or as the authority for) application code.
2. **Implement against the spec** — models, routes, services, and views must trace back to an explicit requirement. If it is not in a feature file, it should not be invented in code.
3. **Verify with tests** — each acceptance scenario maps to automated tests. "Done" means tests pass, not "it looks fine on my machine."
4. **Govern with rules** — stack and team conventions live in `.cursor/rules/` so humans and AI assistants (e.g. Cursor) build the same way.

SDD does **not** mean "never change the code." It means **requirements are the source of truth**, and code + tests prove the requirements.

---

2. What is OC CS Speckit?

OC CS Speckit is Oklahoma Christian's course/kit for practicing SDD with AI-assisted coding in Cursor. It is both:

1. A **methodology package** — feature templates, Cursor rules, ADRs, NFR guidance, living reference docs, and tooling (tests, PDF export, Agility import, deploy scripts).
2. A **Todo example application** — a small multi-user Todo app (auth, lists, items, profile, due dates) that shows the process end to end: specs → rules → Vue/Express code → tests.

Layer	Stack (Todo example)
Frontend	Vue 3, Vuetify 4, Vite, vue-router, axios
Backend	Node.js (ES modules), Express, Sequelize, MySQL
Auth	bcryptjs, JWT, server-side Session table
Tests	Jest + supertest (backend), Vitest (frontend)

The Todo API is mounted at `/todo/`. List and todo data are **per-user**; accessing another user's data returns **404** (not 403).

Think of Speckit as three questions answered in three places:

Question	Where
What must the product do?	`features/`
How must we implement it?	`.cursor/rules/`
Why this architecture / quality bar?	`docs/` (ADRs, NFRs, C4)
What exists on the integrated app **right now**?	`features/reference/`

---

3. Bird's-eye map of the repository

```
todo-speckit/
├── README.md                # Project overview and getting started
├── package.json             # Root scripts (test, PDF, starter zip, reset, …)
├── features/                # Specs = product source of truth
├── .cursor/rules/           # Coding / SDD guardrails for humans + AI
├── docs/                    # Architecture why, NFRs, diagrams, assignments
├── frontend/                # Vue SPA
├── backend/                 # Express API
├── scripts/                 # Tooling (bundles, starter zip, Agility, PDF, …)
├── .github/workflows/       # CI test + deploy pipelines
├── dist/                    # Generated artifacts (e.g. starter zip output)
└── node_modules/            # Root tooling deps (not the app runtimes)
```

Each app also has its own `package.json` and `node_modules/` under `frontend/` and `backend/`.

---

4. `features/` — what to build

This folder is the **product** source of truth for SDD.

Path	Meaning
features/README.md	Feature catalog, order, dependencies, links
features/framework.md	How to write specs (template, FR/SC, DoD, workflow)
features/writing-feature-requirements.md	Student guide: stories, FRs, Gherkin, initial data
features/writing-feature-design.md	Student guide: ownership, API, screens, test map
`features/feature-1-user-auth.md` … `feature-5-….md`	**Todo** feature specs (one file per feature)
features/reference/	**Living reference** — current integrated API / schema / behavior
`features/reference/api.md`	Current REST API under `/todo/`
`features/reference/data-model.md`	Current tables and associations
`features/reference/behavior.md`	Current product rules (ownership, sort, validation, UI)
`features/reference/writing-living-reference.md`	How to update reference in the same PR as code

Feature specs vs living reference:
A `feature-N-*.md` file describes a **change** (what this feature adds or alters).
`features/reference/*` describes **what exists now** after features have been merged (the integrated snapshot).

Todo features (example app):

#	Spec	Delivers
1	User auth	Register, login, logout, session, route guards
2	Todo lists	List CRUD, dashboard sidebar
3	Todo items	Item CRUD, dashboard main panel
4	User profile	Profile edit
5	Due dates	Optional due dates, overdue display

Implement in order 1 → 2 → 3; then 4 and 5 (either order after 3).

---

5. `.cursor/rules/` — how to build

Cursor loads these `.mdc` rules so Agent/Chat follow the same conventions as the course.

Rule	Role
constitution.mdc	Global laws: specs first, branching, testing, stack, human as tech lead
feature-branch.mdc	Product code only on `feature/N-…`, not on `main` / `dev`
project-structure.mdc	Folder layout, env vars, ports
api-conventions.mdc	REST shape, JSON responses, Sequelize patterns
auth-patterns.mdc	Login, session, JWT flow
security.mdc	Password hashing, user-scoped data, 404 for cross-user
frontend-services.mdc	Axios client, services layer (no axios in views)
ui-style-system.mdc	Vuetify 4 defaults
testing-standards.mdc	Real tests; no “ghost” assertions
agent-behavior.mdc	Minimal diffs, ask when unclear, verify goals
quality-attributes.mdc	How to read Accepted vs Deferred NFRs

Branch roles (from the constitution):

Branch	Meaning
`main`	Scaffold / starter baseline — keep feature product merges out
`dev`	Integration branch — completed features merge here
`feature/N-short-name`	Work for one feature, branched from `dev`

6. `docs/` — why, quality, diagrams, course materials

Path	Meaning
docs/README.md	Index of everything in `docs/`
docs/adr/	Architecture Decision Records (e.g. client/server, security, MySQL)
`docs/adr/writing-adrs.md`	How to write an ADR
docs/nfr/	Non-functional requirements / quality attributes
`docs/nfr/quality-attributes.md`	App-wide quality bars (Accepted / Deferred / Out of scope)
docs/arch_diagrams/	C4 diagrams (context, container, components, deployment)
docs/agility-import/	Export specs into Digital.ai Agility
docs/STARTER-KIT.md	How to build/use the empty-app starter zip
`docs/ASSIGNMENT-*.md`	Student assignments (this tour, rebuild, starter kit)
`docs/todo-app-specs.md` / `.pdf`	Generated product-focused export
`docs/oc-cs-speckit-specs.md` / `.pdf`	Generated full pack (rules + docs + features)
`docs/ui/`	Optional screen/Figma exports (create as needed)

Generated PDF/Markdown exports are **convenience copies**. Edit the source Markdown (features, rules, ADRs), then regenerate.

---

7. `frontend/` — Vue SPA

Path	Meaning
`frontend/package.json`	Frontend deps and scripts (`dev`, `test`, `build`)
`frontend/src/main.js`	App bootstrap
`frontend/src/App.vue`	Root shell
`frontend/src/router.js`	Routes and navigation guards
`frontend/src/views/`	Route-level pages (Login, Register, Dashboard, …)
`frontend/src/components/`	Reusable UI (e.g. MenuBar)
`frontend/src/services/`	Axios API modules (`*Services.js`) — views call these
`frontend/src/config/`, `utils/`, `plugins/`	Helpers, Vuetify, localStorage utils
`frontend/tests/`	Vitest unit/component tests
`frontend/public/`	Static assets (e.g. `.htaccess` for deploy)
`frontend/vite.config.js`	Vite / dev server (port **8082**)

---

8. `backend/` — Express API

Path	Meaning
`backend/package.json`	Backend deps and scripts
`backend/server.js`	HTTP server entry; mounts API (Todo: `/todo/`)
`backend/app/models/`	Sequelize models (`user`, `session`, `list`, `todo`, …)
`backend/app/controllers/`	Request handlers
`backend/app/routes/`	Express routers; `index.js` wires them
`backend/app/authorization/`	`authenticate` middleware, access helpers
`backend/app/config/`	DB, auth secret, logger, Sequelize instance
`backend/app/utils/`	Small helpers (e.g. due dates)
`backend/tests/`	Jest + supertest API/auth tests
`backend/.env.example`	Template for local secrets (copy to `.env`)
`backend/.env.test.example`	Template for test DB (copy to `.env.test`)

Dev API default port: **3200**.

---

9. Tooling and other top-level files

Path	Meaning
Root package.json	Orchestration: `npm test`, `specs:pdf`, `starter:zip`, `reset:example`, Agility, bundles
scripts/	Node scripts that implement those tools
`scripts/starter-kit/`	Overlay used to build the empty starter zip
`.github/workflows/`	CI: run tests; deploy frontend static + backend on `dev` (when configured)
`.gitignore`	Keeps secrets and `node_modules` out of git
`.env.agility.example`	Optional Agility API credentials template
`dist/`	Output of zip/bundle scripts (often gitignored or regenerated)

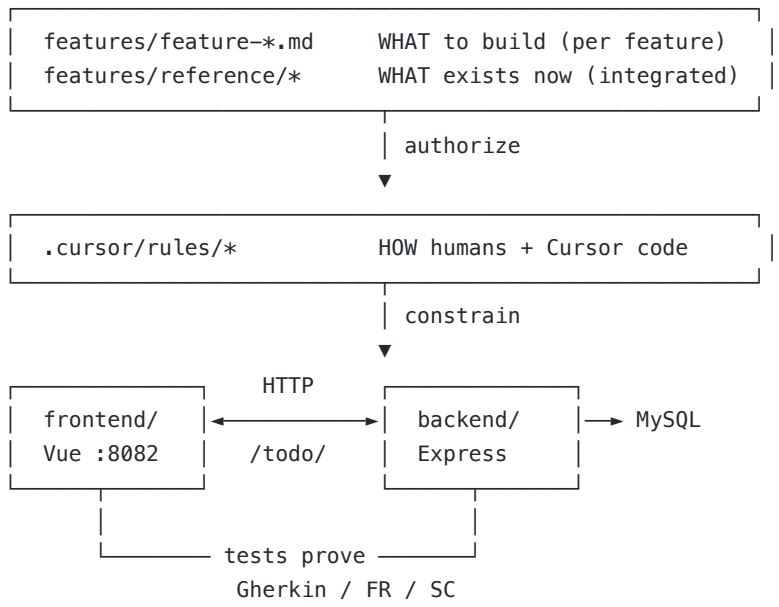
Useful root scripts (conceptual):

Script	Purpose
`npm test`	Backend + frontend tests
`npm run reset:example`	Strip Todo **code**, keep Todo **specs** (for rebuild lab)
`npm run starter:zip`	Build empty-app zip (**no** Todo specs) for a new product

`npm run specs:pdf` / `specs:pdf:app`	Export Markdown/PDF packs

---

10. How the pieces fit (one picture)



`docs/adr + nfr + arch_diagrams` → WHY this shape and quality bar

---

11. Check your understanding

Answer in your own words (a few bullets each):

1. What is SDD, and how does it differ from “code first, document later”?
2. What is OC CS Speckit (methodology + Todo example)?
3. Where do you look for **what** vs **how** vs **why** vs **what exists now**?
4. What is the difference between a `feature-N-*.md` file and `features/reference/api.md`?
5. What are `main`, `dev`, and `feature/*` for?
6. Name one file under `frontend/` and one under `backend/` and what each is for.
7. When would you use `reset:example` vs `starter:zip`?

---

12. Where to go next

If you want to...	Open
Run the Todo app	Root README.md → Getting started
Rebuild Todo from specs	ASSIGNMENT-rebuild-todo.md
Start your own product	ASSIGNMENT-starter-kit.md + STARTER-KIT.md
Write a new feature properly	features/framework.md